VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Subject: COMPUTER ARCHITECTURE

ASSIGNMENT REPORT

BATTLESHIP

Advisor(s): Phạm Quốc Cường
Student(s): Nguyễn Minh Khôi ID: 2252377

HO CHI MINH CITY, DECEMBER 2023

Contents

1 Introduction

Battleship is a strategic guessing game involving two players. Each player possesses their own grid where they discreetly position a fleet of ships on a 7x7 grid board. The gameplay involves the two players taking turns to guess and target the opponent's ships with the objective of sinking them. The winner of the game is the first player to successfully destroy all of the opponent's ships.
The assignment task involves writing MIPS assembly language code to create a text-based version of the Battleship game intended for two players. The implementation of this code has been executed in the MARS MIPS simulator. The primary goal of this report is to elaborate on the concept and approach used in developing this MIPS assembly language program for the Battleship game.

2 Calculation of Grid Cell Addresses

2.1 Idea

The offset to the address of a cell placed at row R and column C in the grid is calculated using the formula $(R \times 7 + C) \times 4$. This formula determines the position of a cell within the grid's memory allocation, considering a grid of size 7x7 where each cell is represented by an integer that occupies 4 bytes.
Subsequently, the address of the cell is obtained by adding this offset to the base address of the grid. This process enables the program to precisely locate and manipulate individual cells within the grid structure.

2.2 Explanation

Memory Offset :

- **Formula:** $(R \times 7 + C) \times 4$
- **Purpose:** This formula calculates the memory offset for a cell within a grid based on its row and column indices.
- **Components:**
  - R represents the row index.
  - C represents the column index.

  - 4 is the number of bytes occupied by each cell (assuming each cell is represented by an integer occupying 4 bytes).

- **Position Calculation:** The formula $R \times 7 + C$ computes a unique numerical position for a cell within a 7x7 grid. The product $R \times 7$ accounts for the number of cells (rows) that come before the designated row R, and C specifies the additional columns within that row.

- **Multiplying by 4:** The resulting numerical position is then multiplied by 4 to convert it into a byte-based offset. This multiplication by 4 accounts for the fact that each cell in the grid occupies 4 bytes of memory.

Address Calculation:

- **Address Offset:** This offset represents the distance (in bytes) of the targeted cell from the base address of the grid in memory.

- **Adding to Base Address:** By adding this offset to the base address of the grid, the program can precisely locate the memory address of the specific cell within the grid structure.

- **Locating Cells:** This process enables the program to manipulate and access individual cells within the grid by determining their precise memory locations.

3 Setup

3.1 Initialization of 7x7 Grid

At the start of the game, each player possesses a grid board consisting of 7 rows and 7 columns filled with only 0s. The rows and columns are labeled with numbers ranging from 0 to 6. The 7x7 grid is stored by utilizing an array of 7 arrays, each holding 7 integers. Each cell's value is represented by an integer that occupies 4 bytes in memory.

3.2 Ship Placement Mechanism

Each player has six ships to place on their grid, including three ships of length 2, two ships of length 3, and one ship of length 4. The cells that are part of the ship will be converted to 1. A ship location is indicated by the coordinates of the bow and the stern of the ship horizontally or vertically, but not diagonally. Ships cannot overlap.

3.2.1 Player Input for Ship Placement

Players are requested to input four parameters: bow row, bow column, stern row, and stern column, dictating the placement of the ship.

3.2.2 Validation Process

Checking Parameter Ranges The code validates the user input to ensure parameters are within the range of 0 to 6. Any parameter falling outside this range prompts the player to re-enter the parameters.

Verifying Ship Validity Ship validity is verified based on its length and orientation (horizontal or vertical). After the player enters four parameters, where each number is between 0 and 6, the validity of the ship placement will be checked. For instance, for a ship of length $n \times 1$, where n can be 2, 3, or 4:

- For a horizontal ship:
  - The ship is considered valid if the bow row is equal to the stern row and the absolute value of the difference between the bow column and the stern column equals $n - 1$.
- For a vertical ship:
  - The ship is considered valid if the bow column is equal to the stern column and the absolute value of the difference between the bow row and the stern row equals $n - 1$.
- If the ship placement does not meet these conditions, the player will be prompted to re-enter the parameters.

Preventing Ship Overlaps The program checks for ship overlaps by inspecting cells from the bow to stern. For valid ships, the following steps are executed to ensure there is no overlap:

- Check the cells from the coordinates of the ship's bow to the coordinates of the ship's stern.
- If the cell contains a value of 1, it indicates that there is a ship in that location, prompting an overlap notification and forcing the player to re-enter the parameters.

- If there is no ship in any of the cells between the bow and stern. Move from the coordinates of the ship's bow to the cell of the ship's stern. Change the value of these cells to 1, representing the placement of the ship.

3.3 Logic Outline

In the MIPS assembly code, implement the setup phase logic as follow:

1. Initialize a 7x7 grid for each player by creating an array of 7 arrays, each containing 7 integers initialized to '0'.

2. Ask a player to enter coordinates for 3 2x1 ships, 2 3x1 ships and 1 4x1 ship. Once that player has finished, the other player will do the same.
   For each ship, request each player inputs for bow row, bow column, stern row, and stern column respectively.

3. Prompt re-enter if any parameters are not integers within the range of 0 to 6.

4. For a ship of length $n \times 1$, where n can be 2, 3, or 4

   - If bow row = stern row *and* |bow column - stern column| $= n - 1$, that is a horizontal ship
   - If bow column = stern column *and* |bow row - stern row| $= n - 1$, that is a vertical ship

   Prompt re-enter if it not a $n \times 1$ horizontal ship or not a $n \times 1$ vertical ship.

5. Ensure ships do not overlap. Prompt re-enter if any cells from bow to stern on the opponent's grid contain a ship (value '1' stored).

6. If parameters are valid, mark grid cells from bow to stern as '1' to signify ship placement.

7. Terminate the setup phase and move to attack phase when both players have placed all their ships.

4 Attack Phase

After the ship setup, players take turns guessing coordinates on the opponent's grid for an attack. The attack mechanism involves the following steps:

4.1 Player Input and Validation

Each player provides input specifying the row and column to target on the opponent's grid. The program validates the entered coordinates to ensure they fall within the valid range of 0 to 6 for both the row and column. If any of the parameters are invalid, the player is prompted to re-enter the coordinates.

4.2 Attack Execution

When the entered coordinates are valid, the program checks the targeted cell on the opponent's grid.

- If the cell contains a '1', indicating a part of the opponent's ship:
  - The cell value is changed to '0' to mark the hit.
  - The program outputs "HIT!" to notify the player of a successful hit on the opponent's ship.
- If the cell contains a '0', indicating an empty space:
  - The program outputs "Miss!" to inform the player that the attack missed the opponent's ship.

This attack progression continues in alternating turns between players until the game's win condition is met.

4.3 Win Condition

In the context of the game of Battleship, each ship occupies a certain number of cells on the grid based on its size. The statement indicates the ship composition for each player: 3 ships of size 2x1, 2 ships of size 3x1, and 1 ship of size 4x1.
Here's a breakdown:

- 3 ships of size 2x1 will occupy a total of $3 \times 2 = 6$ cells.
- 2 ships of size 3x1 will occupy a total of $2 \times 3 = 6$ cells.
- 1 ship of size 4x1 will occupy a total of $1 \times 4 = 4$ cells.

When summed together: $6 + 6 + 4 = 16$ cells in total will be part of the ships on the grid for each player.
The game's objective is to guess and hit the opponent's ships. Therefore, if a player is informed "HIT!" a total of 16 times during the game, it implies that they have successfully targeted and hit all the cells occupied by the opponent's ships. Thus, the player who achieves this by hitting all 16 cells has effectively sunk all the opponent's ships, leading to victory in the game.

4.4 Logic Outline

In the MIPS assembly code, implement each attack turn logic as follows:

1. Take player input for row and column.
2. Validate the input for each parameter to ensure it falls within the range of 0 to 6.
3. When parameters are valid:
   - Calculate the grid cell address based on the user's input.
   - Retrieve the value stored in that cell on the opponent's grid.
   - Update the cell value based on the hit or miss:
     - If the value is '1', change it to '0' and output "HIT!".
     - If the value is '0', output "Miss!".
4. Repeat the attack phase in alternating turns until one of the two players is notified "HIT!" 16 times and becomes the winner.

5 Conclusion

The creation of a text-based Battleship game in MIPS assembly language has offered a practical exercise in applying computer architecture concepts. The implementation highlighted the utilization of memory management and data representation within the MIPS architecture. The initialization of the grid and ship placement mechanisms relied on memory allocation and manipulation techniques, emphasizing the role of arrays and memory offset calculations in managing game data. The input validation processes and error handling were employed to maintain data integrity and prevent memory access violations. By handling user inputs, updating grid cells, and implementing conditional logic for hit or miss outcomes, this project provided hands-on experience with assembly programming.

References

[1] Wikipedia: Battleship (game)

[2] Youtube: MIPS Tutorial 37 - 2D Arrays